

Documento Base do Sistema Fênix Vôlei

Requisitos funcionais e não funcionais, estudo de viabilidade, telas, cores e estrutura de banco de dados

Versão 1.1 | 23/07/2026

Item	Definição
Projeto	Aplicação web para gestão do time de vôlei Fênix.
Stack escolhida	Front-end: HTML + SCSS + JavaScript com Next.js. Back-end: Node.js. Banco de dados: PostgreSQL.
Objetivo principal	Centralizar inscrições, atletas, mensalidades, seletivas, produtos e relatórios em uma plataforma online 24h.
Público-alvo	Diretoria, atletas, interessados em seletivas, responsáveis e compradores de produtos.
Escopo inicial sugerido	MVP com site público, inscrição, login, painel admin, cadastro de atletas, mensalidades, produtos e pedidos.
Ampliação solicitada	Inclusão do Caixa Fênix, integração com parceiros, loja com carrinho/favoritos e remessas ocultas, fórum, notícias e configurações do time com logos.

1. Objetivo da aplicação

A aplicação Fênix Vôlei tem como objetivo funcionar como uma plataforma web oficial para o time, reunindo em um único ambiente os processos administrativos, financeiros, comerciais e de comunicação com atletas e interessados.

O sistema deve permitir que a diretoria controle inscrições de atletas, mensalidades, vendas de produtos, seletivas e cadastros, mantendo dados organizados e acessíveis pela internet durante 24 horas por dia.

Objetivo específico	Descrição
Profissionalizar a gestão	Reduzir controles manuais em planilhas, mensagens soltas e anotações informais.
Facilitar inscrições	Permitir que interessados preencham formulários de inscrição em seletivas ou cadastro geral.
Controlar mensalidades	Registrar cobranças, vencimentos, pagamentos, atrasos e histórico financeiro do atleta.
Organizar vendas	Cadastrar produtos, controlar estoque, registrar pedidos e acompanhar pagamentos.
Dar transparência ao atleta	Permitir que o atleta consulte sua situação, dados, mensalidades e pedidos.
Apoiar decisões	Gerar relatórios para a diretoria sobre atletas, inadimplência, inscrições e vendas.
Profissionalizar o financeiro	Criar o Caixa Fênix para controlar entradas, saídas, pagamentos, comprovantes, conciliação e integração com mensalidades e vendas.
Ampliar parcerias	Permitir cadastro de parceiros para venda de itens na loja e oferta de serviços, como fisioterapia, com possibilidade de solicitação ou agendamento.
Apoiar vendas por remessa	Permitir produtos com estoque normal ou campanhas ocultas por período, como uniformes enviados para malharia após encerramento das vendas.
Melhorar comunicação	Disponibilizar fórum e cadastro de notícias para divulgar amistosos, seletivas, comunicados e avisos na página inicial.

2. Escopo da solução

2.1 Módulos principais

- Site público: página institucional do Fênix, apresentação do time, chamadas para seletivas e loja.
- Inscrições: formulário público para seletivas, interessados e novos atletas.
- Autenticação: login com permissões para atleta, diretoria e administrador.
- Gestão de atletas: cadastro, edição, status, equipe, categoria, posição e histórico.
- Financeiro: planos, mensalidades, vencimentos, pagamentos e inadimplência.
- Loja: produtos, categorias, estoque, pedidos e itens vendidos.
- Relatórios: visão de atletas, inscrições, mensalidades, vendas e pendências.
- Configurações: dados do time, cores, contatos, textos públicos e parâmetros financeiros.
- Caixa Fênix: controle profissional de entradas, saídas, comprovantes, categorias, conciliação e vínculo com mensalidades/vendas.
- Parceiros: cadastro de parceiros, produtos de parceiros na loja e profissionais/serviços para consulta ou agendamento.
- Loja avançada: produtos com estoque imediato, carrinho, favoritos e produtos por remessa oculta com período de venda.
- Comunicação: fórum, notícias e publicações exibidas também na página inicial.
- Configurações do time: dados institucionais, contatos e logos oficiais por contexto de uso.

2.2 Fora do escopo inicial

- Aplicativo mobile nativo para Android/iOS. A primeira versão será responsiva no navegador.
- Integração obrigatória com pagamento automático. O MVP pode começar com registro manual de pagamentos/PIX.
- Controle avançado de treino, scout e estatísticas de jogo. Pode entrar em fase futura.
- Marketplace aberto sem curadoria e sem controle da diretoria. A primeira integração com parceiros deve ser restrita a parceiros cadastrados e aprovados.

3. Tecnologias escolhidas

Camada	Tecnologia	Justificativa
Front-end	HTML + SCSS + JavaScript + Next.js	Boa base visual, componentização, rotas, SEO para páginas públicas e facilidade de hospedagem em serviços modernos.
Back-end	Node.js	Mesma linguagem do front-end, boa integração com APIs, autenticação, pagamentos e serviços de hospedagem.
Banco de dados	PostgreSQL	Banco relacional robusto, confiável para financeiro, vendas, chaves estrangeiras e relatórios.
API	REST inicialmente	Mais simples para iniciar. Pode evoluir para GraphQL apenas se houver necessidade.
Autenticação	JWT ou sessão segura com cookies HttpOnly	Permite separar usuários por perfil e proteger o painel administrativo.
Hospedagem	Next.js em serviço compatível + API Node + PostgreSQL gerenciado ou VPS	Stack popular e aceita em hospedagens modernas. Evitar hospedagem compartilhada limitada apenas a PHP.

4. Requisitos funcionais

Código	Requisito	Descrição	Prioridade
RF-001	Exibir site público	O sistema deve exibir página inicial com identidade visual do Fênix, apresentação do time, botões para inscrição, loja e contato.	Alta
RF-002	Exibir variações do time	O sistema deve permitir páginas ou filtros para time masculino, feminino e outras categorias futuras.	Média
RF-003	Cadastrar interessado	O sistema deve permitir que uma pessoa interessada preencha nome, contato, data de nascimento, posição e observações.	Alta
RF-004	Cadastrar inscrição em seletiva	O sistema deve permitir inscrição em uma seletiva ativa e guardar status: nova, em análise, aprovada, recusada ou compareceu.	Alta
RF-005	Criar conta de usuário	O sistema deve permitir criação de conta com e-mail, senha e perfil definido pelo administrador ou fluxo de aprovação.	Alta
RF-006	Autenticar usuário	O sistema deve permitir login, logout e recuperação de senha.	Alta
RF-007	Controlar permissões	O sistema deve diferenciar permissões de administrador, diretoria, atleta e visitante.	Alta
RF-008	Cadastrar atleta	O sistema deve permitir cadastro completo de atleta com dados pessoais, contato, categoria, posição e status.	Alta
RF-009	Editar atleta	O sistema deve permitir editar dados cadastrais e registrar alterações importantes.	Alta
RF-010	Vincular atleta a equipe	O sistema deve permitir associar atleta a uma ou mais equipes/categorias ao longo do tempo.	Alta
RF-011	Cadastrar planos de mensalidade	O sistema deve permitir criar planos, valores, dia de vencimento e situação ativa/inativa.	Alta
RF-012	Gerar mensalidades	O sistema deve gerar cobranças por atleta conforme plano, mês de referência e data de vencimento.	Alta

4.1 Requisitos funcionais - continuação

Código	Requisito	Descrição	Prioridade
RF-013	Registrar pagamento	O sistema deve registrar pagamentos manuais ou futuramente automáticos, com método, valor, data e comprovante/observação.	Alta
RF-014	Consultar inadimplência	O sistema deve listar mensalidades abertas, atrasadas e pagas, com filtros por mês, atleta e equipe.	Alta
RF-015	Cadastrar produtos	O sistema deve permitir cadastro de produtos com nome, descrição, preço, categoria, estoque e imagem.	Alta
RF-016	Controlar estoque	O sistema deve reduzir estoque quando pedido for confirmado e permitir ajuste manual pela diretoria.	Média
RF-017	Registrar pedidos	O sistema deve permitir pedido de produtos por usuário logado ou visitante identificado.	Alta
RF-018	Acompanhar pedidos	O sistema deve mostrar status do pedido: criado, aguardando pagamento, pago, separado, entregue ou cancelado.	Alta
RF-019	Área do atleta	O atleta deve consultar dados, mensalidades, pagamentos, pedidos e inscrições realizadas.	Alta
RF-020	Painel administrativo	A diretoria deve visualizar cards com total de atletas, mensalidades abertas, inscrições novas e vendas do mês.	Alta
RF-021	Relatórios	O sistema deve gerar relatórios simples de atletas, financeiro, vendas e inscrições.	Média
RF-022	Notificações	O sistema deve registrar notificações internas sobre cobrança, inscrição aprovada ou pedido atualizado.	Média
RF-023	Configurações do sistema	O administrador deve editar informações públicas, contatos, links sociais e parâmetros do time.	Média
RF-024	Auditoria	O sistema deve guardar logs de ações sensíveis como alteração de pagamento, exclusão de produto e mudança de status de atleta.	Alta
RF-025	Criar Caixa Fênix	O sistema deve possuir um módulo financeiro central chamado Caixa Fênix, com visão de saldo, entradas, saídas, pendências e movimentações por período.	Alta
RF-026	Registrar entradas e saídas	A diretoria deve registrar receitas e despesas manuais, informando valor, data, categoria, conta, responsável, descrição e status.	Alta
RF-027	Categorizar financeiro	O sistema deve permitir categorias financeiras como mensalidade, venda de produto, parceiro, material esportivo, aluguel, taxa, transporte e outros.	Alta
RF-028	Anexar comprovantes	O sistema deve permitir anexar comprovantes de pagamento, recibos, notas, imagens ou PDFs vinculados a pagamentos e movimentações do Caixa Fênix.	Alta
RF-029	Integrar caixa com mensalidades e vendas	Pagamentos confirmados de mensalidades e pedidos devem gerar ou vincular automaticamente uma entrada no Caixa Fênix.	Alta
RF-030	Fechamento e conciliação	O sistema deve permitir conferência de saldo por conta, fechamento de caixa por período, observações e registro do responsável pela conferência.	Média
RF-031	Cadastrar parceiros	A diretoria deve cadastrar parceiros, contatos, tipo de parceria, status, comissão ou observações comerciais.	Alta
RF-032	Vender produtos de parceiros	Produtos da loja podem ser próprios do Fênix ou vinculados a parceiros, mantendo identificação de origem, regras de repasse e controle de pedido.	Alta
RF-033	Cadastrar profissionais parceiros	O sistema deve permitir cadastrar profissionais parceiros, como fisioterapeuta, nutricionista ou preparador físico, com serviços, valores e contatos.	Média
RF-034	Solicitar ou agendar serviço parceiro	Usuários devem poder solicitar atendimento ou agendamento com profissional parceiro, ficando a confirmação sob controle da diretoria ou do parceiro.	Média
RF-035	Definir tipo de venda do produto	Cada produto deve ser configurado como venda por estoque imediato ou venda por remessa/campanha oculta.	Alta
RF-036	Controlar remessa oculta	Produtos por remessa devem possuir data de abertura, data de encerramento, status da compra com fornecedor/malharia, previsão de chegada e retirada.	Alta
RF-037	Carrinho de compras	Na loja, o usuário deve poder adicionar produtos ao carrinho, revisar itens, quantidades, variações e total antes de finalizar o pedido.	Alta
RF-038	Favoritos da loja	Na loja, o usuário logado deve poder favoritar produtos para acessar depois; favoritos aparecem apenas no contexto da loja.	Média
RF-039	Configurar logos do time	Nas configurações do time, a diretoria deve cadastrar ou trocar logo oficial, logo masculino, logo feminino e logo de diretoria quando necessário.	Alta
RF-040	Fórum do time	O sistema deve possuir tela de fórum para comunicados, tópicos, comentários e interação controlada entre atletas, diretoria e público conforme permissão.	Média
RF-041	Cadastrar notícias	A diretoria deve cadastrar notícias e comunicados com título, texto, data, horário, local, imagem, status e destaque.	Alta
RF-042	Publicar notícia na página inicial e fórum	Notícias marcadas como públicas devem aparecer na página inicial e também no fórum/comunicados, quando configurado.	Alta

Código	Requisito	Descrição	Prioridade
RF-043	Moderar conteúdo	A diretoria deve poder editar, ocultar ou remover tópicos, comentários e notícias inadequadas, mantendo registro em auditoria.	Alta
RF-044	Avisar chegada e retirada de remessa	Quando uma remessa for recebida, o sistema deve permitir notificar compradores sobre chegada, local e prazo de retirada.	Média
RF-045	Relatórios do Caixa Fênix	O sistema deve gerar relatórios de saldo, entradas, saídas, vendas, mensalidades, despesas por categoria, parceiros e comprovantes pendentes.	Alta

5. Requisitos não funcionais

Código	Requisito	Descrição	Prioridade
RNF-001	Responsividade	O sistema deve funcionar bem em celular, tablet e desktop, priorizando uso mobile para inscrições e consultas.	Alta
RNF-002	Disponibilidade	A aplicação deve ser hospedada em ambiente online e ficar disponível continuamente, salvo manutenções.	Alta
RNF-003	Segurança de senha	Senhas devem ser armazenadas com hash seguro, nunca em texto puro.	Alta
RNF-004	LGPD e privacidade	Dados pessoais devem ter acesso restrito, finalidade clara e proteção contra vazamento.	Alta
RNF-005	Controle de acesso	Rotas administrativas devem ser protegidas por autenticação e perfil de usuário.	Alta
RNF-006	Backup	O banco de dados deve ter rotina de backup periódico antes de uso real em produção.	Alta
RNF-007	Performance	Páginas principais devem carregar rapidamente e consultas devem usar índices em campos importantes.	Média
RNF-008	Escalabilidade	A arquitetura deve permitir crescimento de usuários, atletas, pedidos e dados financeiros sem troca completa da stack.	Média

5.1 Requisitos não funcionais - continuação

Código	Requisito	Descrição	Prioridade
RNF-009	Manutenibilidade	Código deve ser organizado por módulos, com nomes claros e separação entre front, back e banco.	Alta
RNF-010	Padronização visual	O sistema deve seguir cores, botões, cards e tipografia definidos para a marca Fênix.	Média
RNF-011	Compatibilidade	O sistema deve funcionar nos navegadores modernos mais usados, como Chrome, Edge, Firefox e Safari.	Média
RNF-012	Integridade de dados	O banco deve usar chaves primárias, estrangeiras e restrições para evitar cadastros inconsistentes.	Alta
RNF-013	Observabilidade	Erros importantes devem ser registrados no servidor para manutenção futura.	Média
RNF-014	Usabilidade	Fluxos principais devem exigir poucos passos: inscrição, pagamento, compra e consulta de status.	Alta
RNF-015	SEO básico	Páginas públicas devem ter título, descrição e estrutura adequada para compartilhamento e busca.	Baixa
RNF-016	Portabilidade	A stack deve permitir migração entre hospedagens modernas sem depender de tecnologia exclusiva.	Média
RNF-017	Rastreabilidade financeira	Toda movimentação financeira sensível deve registrar usuário responsável, data, origem, alteração e histórico de auditoria.	Alta
RNF-018	Armazenamento de anexos	Comprovantes e imagens devem ser armazenados em serviço seguro ou pasta controlada, com validação de tipo, tamanho e permissão de acesso.	Alta
RNF-019	Consistência transacional	Confirmações de pagamento, baixa de estoque, criação de pedido e movimentação de caixa devem ser tratadas de forma consistente no back-end.	Alta
RNF-020	Moderação e segurança social	Fórum, notícias e comentários devem possuir controle contra spam, conteúdo inadequado e abuso de permissão.	Média
RNF-021	Configuração sem deploy	Alterações de logo, contatos, links e textos públicos do time devem ser feitas pelo painel sem necessidade de alterar código.	Média
RNF-022	Separação de responsabilidades	Módulos de caixa, loja, parceiros e fórum devem ser independentes o suficiente para manutenção e evolução por etapas.	Média

6. Estudo de viabilidade

Dimensão	Análise	Resultado
Técnica	A stack Next.js + Node.js + PostgreSQL é adequada para sistemas web com login, painel administrativo, formulários, loja e financeiro. Também é uma combinação comum para deploy em serviços modernos.	Viável
Operacional	A diretoria poderá substituir controles manuais por painéis e relatórios. O principal cuidado será treinamento inicial e padronização dos cadastros.	Viável
Econômica	É possível iniciar com custos baixos usando hospedagem em camadas gratuitas ou planos de entrada, evoluindo para VPS ou serviços pagos conforme o uso aumentar.	Viável
Legal/LGPD	O sistema lidará com dados pessoais. É necessário limitar acesso, coletar apenas o necessário, proteger senhas e ter política simples de privacidade.	Viável com cuidados
Cronograma	Um MVP pode ser planejado em fases: base visual, autenticação, inscrições, atletas, mensalidades, produtos e relatórios.	Viável em fases
Manutenção	Como JavaScript será usado no front e back, a manutenção tende a ser mais simples para uma equipe pequena.	Viável
Financeira/controle interno	O Caixa Fênix aumenta a confiabilidade dos registros, mas exige processo mínimo de conferência, categorias padronizadas e política de anexos.	Viável com processo
Parcerias	A venda de itens e serviços de parceiros é viável, desde que a diretoria aprove os parceiros e defina regras claras de repasse, responsabilidade e atendimento.	Viável com curadoria
Comunicação	Fórum e notícias melhoram divulgação de amistosos e eventos, mas precisam de moderação e perfis de permissão para evitar uso inadequado.	Viável com moderação

6.1 Riscos e mitigação

Risco	Impacto	Mitigação
Tentar construir tudo de uma vez	Atraso e abandono do projeto	Criar MVP com funções essenciais: inscrição, atletas, mensalidade e loja básica.
Dados financeiros inconsistentes	Relatórios errados e cobrança duplicada	Usar regras no banco, status claros e logs de auditoria.
Vazamento de dados pessoais	Problema legal e perda de confiança	Aplicar controle de acesso, senhas com hash, backups seguros e permissão mínima.
Hospedagem incompatível	Dificuldade para colocar no ar	Evitar hospedagem compartilhada limitada. Usar serviço compatível com Node.js e PostgreSQL.
Dependência de pagamento automático no início	Complexidade extra	Começar com PIX/manual e preparar tabela de pagamentos para integração futura.
Caixa sem conferência periódica	Saldo incorreto e perda de confiança nos relatórios	Definir fechamento mensal, responsáveis, categorias obrigatórias e anexos para despesas importantes.
Produtos por remessa mal configurados	Vendas fora do prazo, pedido errado para malharia e reclamações	Exigir datas de abertura/encerramento, status da remessa, tamanhos/variações e relatório antes do pedido ao fornecedor.
Parceiro sem regra de repasse	Conflito financeiro ou operacional	Cadastrar tipo de parceria, comissão/repasse, responsável e observações contratuais antes de vender itens de parceiro.
Fórum sem moderação	Mensagens indevidas ou ruído na comunicação oficial	Permissões por perfil, opção de ocultar conteúdo, logs de auditoria e canal oficial para notícias importantes.

7. Identidade visual e cores padrão

A identidade principal recomendada deve usar o vermelho, preto e branco do Fênix. A arte verde/amarela da parceria com Floresta deve ficar fora da marca principal, pois a parceria pode acabar.

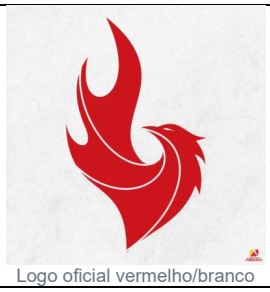
7.1 Logos de referência



Logo masculino



Logo feminino


Logo oficial vermelho/branco


Diretoria preto/branco


Parceria Floresta - não usar como marca principal

Token	Hex	Uso recomendado
--color-primary-red	#B80012	Cor principal: botões, links ativos, destaques, cabeçalho de tabelas e chamadas importantes.
--color-dark-red	#7A000A	Hover, sombras, gradientes e variações mais fortes do vermelho.
--color-black	#121212	Texto principal, sidebar, rodapé e áreas de contraste.
--color-white	#FFFFFF	Fundos limpos, texto sobre vermelho/preto e cards.
--color-gray-bg	#F7F7F7	Fundo de páginas internas e áreas administrativas.
--color-gray-line	#D9D9D9	Bordas, divisórias e linhas de tabela.
--color-success	#16A34A	Status pago, ativo, aprovado.
--color-warning	#F2C94C	Status pendente, aguardando pagamento, análise.

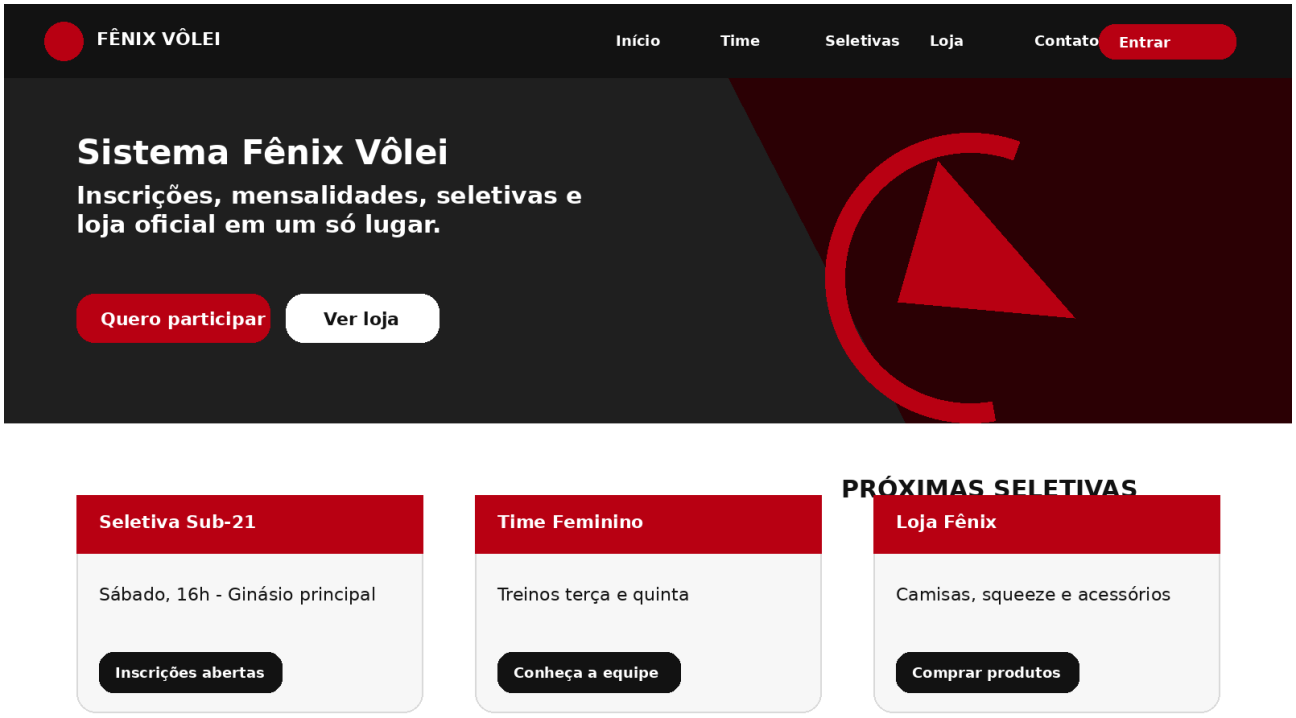
Token	Hex	Uso recomendado
--color-danger	#DC2626	Atraso, erro, cancelado, ação destrutiva.
--color-info	#2563EB	Links informativos, avisos neutros, ações de consulta e elementos de fórum/notícias.
--color-muted	#6B7280	Textos auxiliares, datas, legendas, informações secundárias e status menos críticos.
--color-accent-gold	#F2C94C	Detalhes de destaque, chamadas de seletiva, itens especiais da loja e avisos importantes sem caráter de erro.

7.2 Padrões de interface

- Botão principal: fundo vermelho, texto branco, bordas arredondadas.
- Botão secundário: fundo branco, borda vermelha, texto vermelho.
- Cards: fundo branco ou cinza claro, sombra leve, borda suave.
- Tabelas administrativas: cabeçalho vermelho, linhas alternadas claras e ações no fim da linha.
- Status: usar cores sem depender apenas da cor; sempre incluir texto como Pago, Aberto, Atrasado, Pendente.
- Tipografia sugerida: Inter, Montserrat ou Arial. Usar fonte forte nos títulos e fonte simples nos textos.

8. Telas do sistema - wireframes iniciais

Os desenhos abaixo são wireframes de baixa fidelidade. Eles definem estrutura, hierarquia e navegação antes da criação do layout final no código.



Tela 1 - Página pública inicial

FÊNIX VÔLEI

Formulário público

Inscrição em Seletiva

Dados do interessado / atleta

Nome completo

Data de nascimento

Telefone/WhatsApp

E-mail

Posição

Experiência no vôlei

Enviar inscrição

Acesso do atleta

Entre para consultar mensalidades, pagamentos e pedidos.

E-mail

Senha

Entrar

Criar conta

Tela 2 - Inscrição em seletiva e acesso do atleta

FÊNIX

- Dashboard
- Atletas
- Mensalidades
- Produtos
- Pedidos
- Seletivas
- Relatórios
- Configurações

Painel Administrativo

Visão geral do time, financeiro, loja e seletivas.

Atletas ativos	Mensalidades em aberto	Inscrições novas	Vendas do mês
86	14	22	R$ 1.840

Pendências recentes

Tipo	Nome	Status	Ação
Mensalidade	João Souza	Atrasada	Cobrar
Inscrição	Maria Lima	Nova	Avaliar
Pedido	Camisa oficial	Pago	Separar
Atleta	Pedro Alves	Cadastro incompleto	Editar

Tela 3 - Dashboard administrativo

Módulo: Atletas e Financeiro

Atletas

Buscar atleta...

Novo

Ana Silva - Levantadora	Ativo
Carlos Lima - Central	Ativo
João Souza - Ponteiro	Ativo
Marina Costa - Líbero	Pendente
Pedro Alves - Oposto	Ativo

Mensalidades

Referência: Julho/2026

Pagas	Abertas	Atrasadas
72	14	5

Gerar cobranças mensais

Gerar mensalidades

Lista de cobranças

João Souza	Atrasada
Ana Silva	Paga
Pedro Alves	Aberta

Tela 4 - Gestão de atletas e mensalidades

Módulo: Loja e Seletivas

Produtos

Imagem do produto

Camisa oficial

R$ 89,90

Adicionar

Imagem do produto

Squeeze Fênix

R$ 29,90

Adicionar

Imagem do produto

Meião treino

R$ 24,90

Adicionar

Seletivas / Interessados

Evento	Data	Inscritos	Status	Ação
Seletiva Adulto Masculino	27/07/2026	38	**Aberta**	Ver lista
Seletiva Feminino	03/08/2026	24	**Rascunho**	Publicar

Tela 5 - Loja e seletivas



Tela 6 - Área do atleta

Tela 7 - Caixa Fênix

[Sidebar Admin] Caixa Fênix

Cards: Saldo atual | Entradas do mês | Saídas do mês | Pendências

Filtros: período, conta, categoria, origem, status

Tabela: data | tipo | categoria | descrição | origem | valor | comprovante | ações

Ações: Nova entrada | Nova saída | Anexar comprovante | Fechar caixa | Exportar relatório

Tela 8 - Loja com carrinho e favoritos

[Topo público] Logo | Início | Loja | Fórum | Login

Busca e filtros: categoria, tamanho, estoque, remessa, parceiro

Grid de produtos: imagem | nome | preço | status | favorito | adicionar ao carrinho

Carrinho lateral/página: itens | quantidade | variação | subtotal | finalizar pedido

Produtos por remessa: selo de campanha, datas de abertura/encerramento e previsão de retirada

Tela 9 - Fórum e notícias

[Página Fórum] Destaques | Notícias | Tópicos | Comunicados

Card de notícia: título | data | local | resumo | imagem | botão ver detalhes

Exemplo: Amistoso Fênix contra XXXXX - dia YYYY às XX:XX no local TAL

Tópico: autor | categoria | comentários | status | moderação

Tela 10 - Parceiros e serviços

[Parceiros] lista de parceiros aprovados | tipo | contato | produtos | serviços

[Profissional] fisioterapeuta/nutricionista/etc. | serviços | valor | horários | solicitar/agendar

[Admin] aprovar parceiro | cadastrar produto parceiro | regra de repasse | histórico

Tela 11 - Configurações do time

[Admin > Configurações] Dados do time | Contatos | Redes sociais | Cores | Logos

Upload de logos: oficial | masculino | feminino | diretoria | variações futuras

Pré-visualização: cabeçalho público | card de produto | notícia | rodapé

8.1 Mapa de telas recomendado

Área	Tela	Função principal
Pública	Início	Apresentar o Fênix, mostrar chamadas de inscrição e loja.
Pública	Seletivas	Listar seletivas abertas e formulário de inscrição.
Pública	Loja	Mostrar produtos disponíveis e iniciar pedido.
Pública	Contato	Exibir WhatsApp, redes sociais, localização e informações do time.
Autenticação	Login	Permitir acesso de atleta, diretoria e administrador.
Autenticação	Cadastro	Criar conta ou completar cadastro após aprovação.
Admin	Dashboard	Exibir indicadores principais e pendências.
Admin	Atletas	Cadastrar, editar, filtrar, ativar/inativar e vincular a equipes.
Admin	Financeiro	Gerar mensalidades, registrar pagamento e controlar inadimplência.
Admin	Produtos	Cadastrar produtos, controlar estoque e preços.
Admin	Pedidos	Acompanhar vendas e entregas.
Admin	Seletivas	Criar seletivas, ver inscritos e aprovar interessados.
Admin	Relatórios	Emitir relatórios simples para tomada de decisão.
Atleta	Minha área	Consultar mensalidade, perfil, pedidos e inscrições.
Pública	Fórum	Exibir notícias, comunicados, tópicos e comentários conforme permissão.
Pública	Notícia/Comunicado	Mostrar detalhes de amistosos, eventos, seletivas, local, data, horário e informações adicionais.
Loja	Carrinho	Revisar itens, quantidades, tamanhos/variações, total e dados para finalizar pedido.
Loja	Favoritos	Mostrar produtos favoritados pelo usuário apenas dentro da área da loja.
Admin	Caixa Fênix	Registrar entradas, saídas, anexos, conciliação, fechamento e relatórios financeiros.
Admin	Parceiros	Cadastrar parceiros, produtos de parceiros, profissionais e serviços.
Admin	Remessas	Abrir/encerrar campanhas ocultas, conferir pedidos para malharia e controlar chegada/retirada.
Admin	Notícias	Cadastrar notícias e definir publicação na página inicial e no fórum.
Admin	Configurações do time	Alterar dados institucionais, contatos, links, cores e logos.

9. Estrutura de banco de dados

O banco foi modelado para PostgreSQL usando chaves primárias UUID, chaves estrangeiras e tabelas de relacionamento. A modelagem separa usuários, atletas, equipes, financeiro, loja e inscrições.

Modelo relacional - Fênix Vôlei

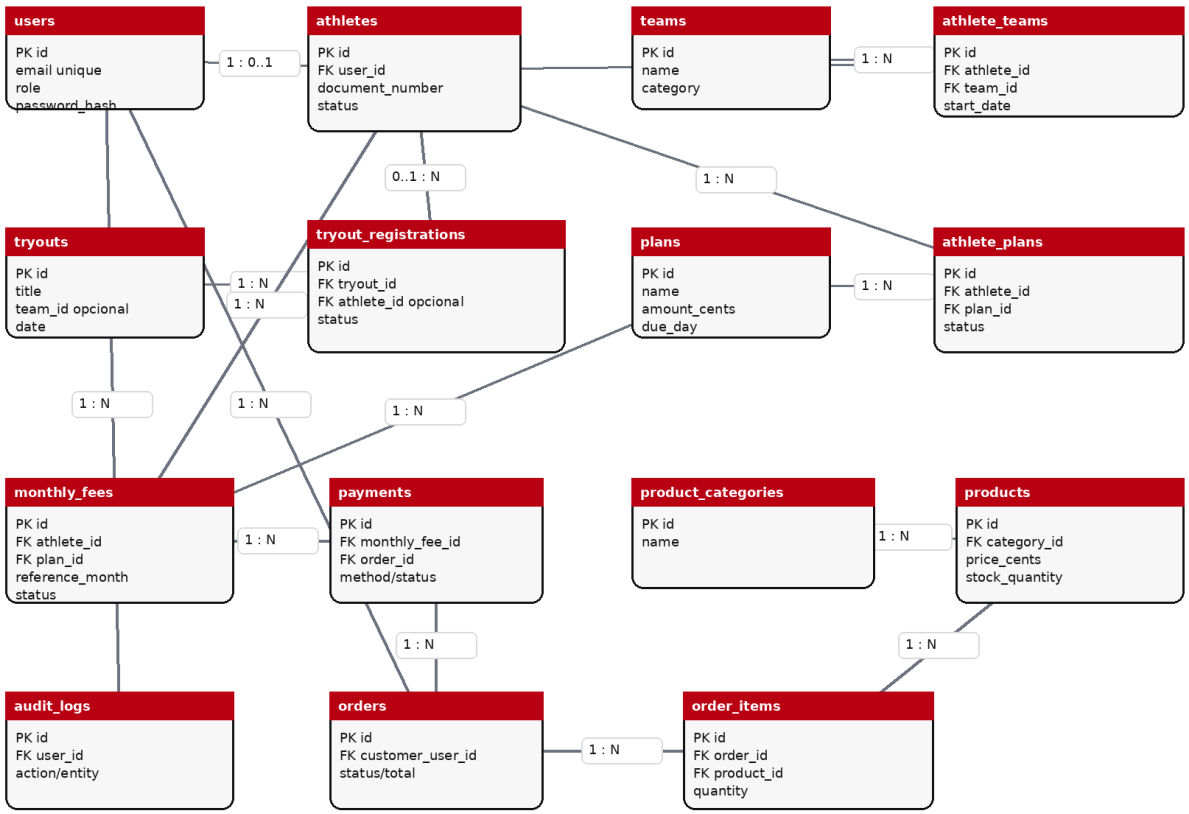


Diagrama simplificado de entidades e relacionamentos

9.1 Tabelas principais e responsabilidades

Tabela	Responsabilidade	Chaves e conexões
users	Armazena contas de acesso ao sistema.	PK: id. Relaciona com athletes, orders, audit_logs e notifications.
athletes	Armazena dados de atleta ou cadastro aprovado.	PK: id. FK opcional: user_id. Relaciona com teams, monthly_fees, athlete_plans e tryout_registrations.
teams	Equipes/categorias do Fênix, como masculino, feminino, base ou misto.	PK: id. Relaciona com athlete_teams e tryouts.
athlete_teams	Tabela N:N para vincular atletas a equipes ao longo do tempo.	PK: id. FK: athlete_id, team_id.
tryouts	Seletivas/eventos de seleção.	PK: id. FK opcional: team_id.
tryout_registrations	Inscrições em seletivas ou interesse em participar.	PK: id. FK: tryout_id. FK opcional: athlete_id.
plans	Planos de mensalidade e valores.	PK: id. Relaciona com athlete_plans e monthly_fees.
athlete_plans	Histórico de plano aplicado a cada atleta.	PK: id. FK: athlete_id, plan_id.
monthly_fees	Cobranças mensais por atleta.	PK: id. FK: athlete_id, plan_id.
payments	Pagamentos de mensalidade ou pedido.	PK: id. FK opcional: monthly_fee_id e order_id.
product_categories	Categorias de produtos da loja.	PK: id. Relaciona com products.
products	Produtos do time, com preço e estoque.	PK: id. FK: category_id.
orders	Pedidos realizados na loja.	PK: id. FK opcional: customer_user_id.
order_items	Itens de cada pedido.	PK: id. FK: order_id, product_id.
audit_logs	Registro de ações administrativas importantes.	PK: id. FK opcional: user_id.
notifications	Notificações internas para usuários.	PK: id. FK: user_id.
cash_accounts	Contas financeiras do Caixa Fênix, como caixa físico, PIX, banco ou carteira.	PK: id. Relaciona com cash_movements e cash_reconciliations.
cash_categories	Plano de categorias para entradas, saídas e transferências.	PK: id. Relaciona com cash_movements.
cash_movements	Movimentações financeiras do Caixa Fênix.	PK: id. FK: account_id, category_id, created_by_user_id. FK opcionais: payment_id, order_id, monthly_fee_id, partner_id.
cash_reconciliations	Fechamentos/conciliações de caixa por conta e período.	PK: id. FK: account_id, closed_by_user_id.
financial_attachments	Comprovantes, recibos e anexos financeiros.	PK: id. FK opcionais: payment_id, cash_movement_id, uploaded_by_user_id.

Tabela	Responsabilidade	Chaves e conexões
partners	Parceiros aprovados do Fênix, fornecedores, lojas, profissionais ou patrocinadores.	PK: id. Relaciona com products, partner_professionals, professional_services, appointments e cash_movements.
partner_professionals	Profissionais vinculados a parceiros.	PK: id. FK: partner_id. Relaciona com professional_services e appointments.
professional_services	Serviços oferecidos por profissionais parceiros.	PK: id. FK: partner_id e professional_id. Relaciona com appointments.
appointments	Solicitações ou agendamentos com profissionais parceiros.	PK: id. FK: service_id, requester_user_id, athlete_id.
product_batches	Remessas/campanhas de venda por período, como uniformes para malharia.	PK: id. FK: product_id, supplier_partner_id. Relaciona com order_items.
inventory_movements	Entradas, saídas e ajustes de estoque.	PK: id. FK: product_id, order_item_id, created_by_user_id.
carts	Carrinho de compras ativo ou convertido em pedido.	PK: id. FK: user_id e converted_order_id. Relaciona com cart_items.
cart_items	Itens adicionados ao carrinho.	PK: id. FK: cart_id, product_id, product_batch_id.
favorite_products	Produtos favoritados pelo usuário na loja.	PK composta sugerida: user_id, product_id. FK: user_id, product_id.
team_settings	Configurações institucionais do time.	PK: id. Guarda nome, contatos, links, cores e logo oficial.
team_logos	Logos por contexto de uso: oficial, masculino, feminino, diretoria e outros.	PK: id. FK opcional: team_id. Relaciona com configurações e telas públicas.
forum_categories	Categorias do fórum e comunicados.	PK: id. Relaciona com forum_topics.
forum_topics	Tópicos do fórum criados por usuários autorizados.	PK: id. FK: category_id, author_user_id. Relaciona com forum_posts.
forum_posts	Comentários/respostas dentro dos tópicos.	PK: id. FK: topic_id, author_user_id.
news_posts	Notícias e comunicados publicados na página inicial e/ou fórum.	PK: id. FK: author_user_id.

9.2 Relacionamentos principais

- Um usuário pode estar ligado a zero ou um atleta; nem todo usuário será atleta, pois existem administradores e diretoria.
- Um atleta pode participar de várias equipes ao longo do tempo, e uma equipe possui vários atletas.
- Uma seletiva pode receber várias inscrições. Uma inscrição pode virar atleta após aprovação.
- Um atleta pode ter um plano ativo e várias mensalidades geradas mês a mês.
- Uma mensalidade pode ter um ou mais pagamentos, permitindo pagamento parcial, correção ou registro de múltiplos métodos.
- Um pedido possui vários itens, e cada item aponta para um produto.
- Pagamentos podem estar ligados a mensalidades, pedidos ou ambos em casos futuros, mas o ideal é manter regras claras no back-end.
- Todo pagamento confirmado de mensalidade ou pedido deve gerar ou vincular uma movimentação do Caixa Fênix.
- Movimentações do Caixa Fênix devem ter conta, categoria, tipo, valor, responsável e podem possuir comprovantes anexados.
- Produtos podem ser próprios do Fênix ou de parceiros aprovados; cada produto pode trabalhar com estoque imediato ou remessa/campanha.
- Remessas controlam período de venda, pedido ao fornecedor/malharia, previsão de chegada e retirada pelos compradores.
- Carrinhos e favoritos pertencem ao usuário; favoritos devem aparecer apenas na experiência da loja.
- Parceiros podem oferecer produtos e também serviços profissionais, como fisioterapia, com solicitações ou agendamentos.
- Notícias podem aparecer na página inicial e no fórum; comentários e tópicos devem respeitar permissões e moderação.
- Configurações do time centralizam dados públicos e logos para evitar alteração manual no código.

9.3 SQL base sugerido

Este SQL é um ponto de partida técnico. Ajustes finos podem ser feitos quando o desenvolvimento iniciar.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(160) NOT NULL,
    email VARCHAR(180) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    phone VARCHAR(30),
    role VARCHAR(30) NOT NULL CHECK (role IN ('admin', 'diretoria', 'atleta')),
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive', 'blocked')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE athletes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID UNIQUE REFERENCES users(id) ON DELETE SET NULL,
    full_name VARCHAR(160) NOT NULL,
    birth_date DATE,
    document_number VARCHAR(30) UNIQUE,
    phone VARCHAR(30),
    email VARCHAR(180),
    gender VARCHAR(20) CHECK (gender IN ('masculino', 'feminino', 'outro', 'nao_informado')),
    position VARCHAR(60),
    emergency_contact_name VARCHAR(160),
    emergency_contact_phone VARCHAR(30),
    status VARCHAR(30) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'active', 'inactive', 'suspended')),
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE teams (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL,
    category VARCHAR(60) NOT NULL,
    description TEXT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE athlete_teams (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id UUID NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,
    team_id UUID NOT NULL REFERENCES teams(id) ON DELETE CASCADE,
    start_date DATE NOT NULL DEFAULT CURRENT_DATE,
    end_date DATE,
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
    UNIQUE (athlete_id, team_id, start_date)
);

CREATE TABLE tryouts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
    title VARCHAR(160) NOT NULL,
    description TEXT,
    location VARCHAR(180),
    event_date TIMESTAMP,
    registration_start TIMESTAMP,
    registration_end TIMESTAMP,
    max_slots INTEGER,
    status VARCHAR(30) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'open', 'closed', 'finished', 'cancelled')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE tryout_registrations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tryout_id UUID NOT NULL REFERENCES tryouts(id) ON DELETE CASCADE,
    athlete_id UUID REFERENCES athletes(id) ON DELETE SET NULL,
    full_name VARCHAR(160) NOT NULL,
    birth_date DATE,
    phone VARCHAR(30) NOT NULL,
```

```
    email VARCHAR(180),
    gender VARCHAR(20),
    position VARCHAR(60),
    experience TEXT,
    status VARCHAR(30) NOT NULL DEFAULT 'new' CHECK (status IN ('new', 'reviewing', 'approved', 'rejected', 'attended',
'no_show')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL,
    amount_cents INTEGER NOT NULL CHECK (amount_cents >= 0),
    due_day INTEGER NOT NULL CHECK (due_day BETWEEN 1 AND 28),
    description TEXT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE athlete_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id UUID NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,
    plan_id UUID NOT NULL REFERENCES plans(id) ON DELETE RESTRICT,
    start_date DATE NOT NULL DEFAULT CURRENT_DATE,
    end_date DATE,
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE monthly_fees (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id UUID NOT NULL REFERENCES athletes(id) ON DELETE CASCADE,
    plan_id UUID NOT NULL REFERENCES plans(id) ON DELETE RESTRICT,
    reference_month DATE NOT NULL,
    due_date DATE NOT NULL,
    amount_cents INTEGER NOT NULL CHECK (amount_cents >= 0),
    status VARCHAR(30) NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'paid', 'overdue', 'cancelled')),
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE (athlete_id, reference_month)
);

CREATE TABLE product_categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL UNIQUE,
    description TEXT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id UUID REFERENCES product_categories(id) ON DELETE SET NULL,
    name VARCHAR(160) NOT NULL,
    description TEXT,
    price_cents INTEGER NOT NULL CHECK (price_cents >= 0),
    stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    image_url TEXT,
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive', 'sold_out')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    customer_name VARCHAR(160) NOT NULL,
    customer_phone VARCHAR(30),
    customer_email VARCHAR(180),
    total_cents INTEGER NOT NULL DEFAULT 0 CHECK (total_cents >= 0),
    status VARCHAR(30) NOT NULL DEFAULT 'created' CHECK (status IN ('created', 'waiting_payment', 'paid', 'separated',
'delivered', 'cancelled')),
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price_cents INTEGER NOT NULL CHECK (unit_price_cents >= 0),
    total_cents INTEGER NOT NULL CHECK (total_cents >= 0)
);

CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    monthly_fee_id UUID REFERENCES monthly_fees(id) ON DELETE SET NULL,
    order_id UUID REFERENCES orders(id) ON DELETE SET NULL,
    payer_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    method VARCHAR(30) NOT NULL CHECK (method IN ('pix', 'cash', 'card', 'bank_transfer', 'other')),
    amount_cents INTEGER NOT NULL CHECK (amount_cents > 0),
    status VARCHAR(30) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'paid', 'cancelled', 'refunded')),
    transaction_code VARCHAR(160),
    paid_at TIMESTAMP,
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (monthly_fee_id IS NOT NULL OR order_id IS NOT NULL)
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(160) NOT NULL,
    message TEXT NOT NULL,
    read_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(120) NOT NULL,
    entity_name VARCHAR(120) NOT NULL,
    entity_id UUID,
    old_data JSONB,
    new_data JSONB,
    ip_address VARCHAR(60),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_athletes_status ON athletes(status);
CREATE INDEX idx_monthly_fees_status_due_date ON monthly_fees(status, due_date);
CREATE INDEX idx_tryout_registrations_status ON tryout_registrations(status);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_payments_status ON payments(status);
```

9.4 Complementos de banco para as novas funcionalidades

As estruturas abaixo complementam o SQL base sem remover as tabelas já definidas. Elas cobrem Caixa Fênix, anexos, parceiros, remessas, carrinho, favoritos, fórum, notícias e configurações do time.

```
-- Complementos profissionais para Caixa Fênix, parceiros, loja avançada, fórum e notícias



CREATE TABLE cash_accounts (

  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  name VARCHAR(120) NOT NULL,

  account_type VARCHAR(30) NOT NULL CHECK (account_type IN ('cash', 'pix', 'bank', 'card', 'other')),

  initial_balance_cents INTEGER NOT NULL DEFAULT 0,

  is_active BOOLEAN NOT NULL DEFAULT TRUE,

  created_at TIMESTAMP NOT NULL DEFAULT NOW()
```

```
);

CREATE TABLE cash_categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL,
    movement_type VARCHAR(30) NOT NULL CHECK (movement_type IN ('income', 'expense', 'transfer')),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    UNIQUE (name, movement_type)
);

CREATE TABLE partners (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(160) NOT NULL,
    partner_type VARCHAR(30) NOT NULL CHECK (partner_type IN ('supplier', 'store', 'professional', 'sponsor', 'other')),
    contact_name VARCHAR(160),
    phone VARCHAR(30),
    email VARCHAR(180),
    document_number VARCHAR(40),
    commission_type VARCHAR(30) NOT NULL DEFAULT 'none' CHECK (commission_type IN ('none', 'percentage', 'fixed')),
    commission_percentage NUMERIC(5,2) DEFAULT 0,
    commission_value_cents INTEGER DEFAULT 0,
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive', 'pending')),
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

ALTER TABLE products ADD COLUMN partner_id UUID REFERENCES partners(id) ON DELETE SET NULL;
ALTER TABLE products ADD COLUMN sale_mode VARCHAR(30) NOT NULL DEFAULT 'stock' CHECK (sale_mode IN ('stock', 'pre_order', 'hidden_batch'));
ALTER TABLE products ADD COLUMN is_hidden BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE products ADD COLUMN sales_start_at TIMESTAMP;
ALTER TABLE products ADD COLUMN sales_end_at TIMESTAMP;
ALTER TABLE products ADD COLUMN requires_size BOOLEAN NOT NULL DEFAULT FALSE;

CREATE TABLE product_batches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    supplier_partner_id UUID REFERENCES partners(id) ON DELETE SET NULL,
    title VARCHAR(160) NOT NULL,
    description TEXT,
    sales_start_at TIMESTAMP NOT NULL,
    sales_end_at TIMESTAMP NOT NULL,
    supplier_order_date DATE,
    expected_arrival_date DATE,
    pickup_start_date DATE,
    status VARCHAR(30) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'open', 'closed', 'ordered', 'received', 'finished', 'cancelled')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (sales_end_at > sales_start_at)
);

ALTER TABLE order_items ADD COLUMN product_batch_id UUID REFERENCES product_batches(id) ON DELETE SET NULL;
ALTER TABLE order_items ADD COLUMN selected_size VARCHAR(40);
ALTER TABLE order_items ADD COLUMN selected_variation VARCHAR(80);

CREATE TABLE inventory_movements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    order_item_id UUID REFERENCES order_items(id) ON DELETE SET NULL,
    created_by_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    movement_type VARCHAR(30) NOT NULL CHECK (movement_type IN ('in', 'out', 'adjustment', 'reserved', 'released')),
    quantity INTEGER NOT NULL,
    reason TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE carts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    session_token VARCHAR(160),
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'converted', 'abandoned')),
    converted_order_id UUID REFERENCES orders(id) ON DELETE SET NULL,
```

```
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE cart_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    cart_id UUID NOT NULL REFERENCES carts(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    product_batch_id UUID REFERENCES product_batches(id) ON DELETE SET NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    selected_size VARCHAR(40),
    selected_variation VARCHAR(80),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE favorite_products (
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, product_id)
);

CREATE TABLE cash_movements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id UUID NOT NULL REFERENCES cash_accounts(id) ON DELETE RESTRICT,
    category_id UUID NOT NULL REFERENCES cash_categories(id) ON DELETE RESTRICT,
    created_by_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    payment_id UUID REFERENCES payments(id) ON DELETE SET NULL,
    order_id UUID REFERENCES orders(id) ON DELETE SET NULL,
    monthly_fee_id UUID REFERENCES monthly_fees(id) ON DELETE SET NULL,
    partner_id UUID REFERENCES partners(id) ON DELETE SET NULL,
    movement_type VARCHAR(30) NOT NULL CHECK (movement_type IN ('income', 'expense', 'transfer_in', 'transfer_out')),
    amount_cents INTEGER NOT NULL CHECK (amount_cents > 0),
    status VARCHAR(30) NOT NULL DEFAULT 'confirmed' CHECK (status IN ('pending', 'confirmed', 'cancelled')),
    description TEXT NOT NULL,
    occurred_at TIMESTAMP NOT NULL DEFAULT NOW(),
```

```
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE financial_attachments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id UUID REFERENCES payments(id) ON DELETE CASCADE,
    cash_movement_id UUID REFERENCES cash_movements(id) ON DELETE CASCADE,
    uploaded_by_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    file_url TEXT NOT NULL,
    file_name VARCHAR(220),
    mime_type VARCHAR(120),
    file_size_bytes INTEGER,
    attachment_type VARCHAR(30) NOT NULL DEFAULT 'receipt' CHECK (attachment_type IN ('receipt', 'invoice', 'contract', 'other')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (payment_id IS NOT NULL OR cash_movement_id IS NOT NULL)
);

CREATE TABLE cash_reconciliations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id UUID NOT NULL REFERENCES cash_accounts(id) ON DELETE RESTRICT,
    closed_by_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    expected_balance_cents INTEGER NOT NULL,
    counted_balance_cents INTEGER NOT NULL,
    difference_cents INTEGER NOT NULL DEFAULT 0,
    notes TEXT,
    closed_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (period_end >= period_start)
);

CREATE TABLE partner_professionals (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID NOT NULL REFERENCES partners(id) ON DELETE CASCADE,
    full_name VARCHAR(160) NOT NULL,
```

```
    profession VARCHAR(120) NOT NULL,
    license_number VARCHAR(80),
    phone VARCHAR(30),
    email VARCHAR(180),
    status VARCHAR(30) NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE professional_services (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID NOT NULL REFERENCES partners(id) ON DELETE CASCADE,
    professional_id UUID REFERENCES partner_professionals(id) ON DELETE SET NULL,
    name VARCHAR(160) NOT NULL,
    description TEXT,
    price_cents INTEGER CHECK (price_cents >= 0),
    duration_minutes INTEGER CHECK (duration_minutes > 0),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE appointments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    service_id UUID NOT NULL REFERENCES professional_services(id) ON DELETE RESTRICT,
    requester_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    athlete_id UUID REFERENCES athletes(id) ON DELETE SET NULL,
    requested_at TIMESTAMP NOT NULL DEFAULT NOW(),
    scheduled_at TIMESTAMP,
    status VARCHAR(30) NOT NULL DEFAULT 'requested' CHECK (status IN ('requested', 'confirmed', 'completed', 'cancelled', 'no_show')),
    notes TEXT
);

CREATE TABLE team_settings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    singleton_key VARCHAR(40) NOT NULL DEFAULT 'main' UNIQUE,
    team_name VARCHAR(160) NOT NULL,
    official_logo_url TEXT,
    primary_color VARCHAR(20),
```

```
    secondary_color VARCHAR(20),
    whatsapp VARCHAR(30),
    instagram_url TEXT,
    address TEXT,
    updated_by_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE team_logos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    context VARCHAR(40) NOT NULL CHECK (context IN ('official', 'masculino', 'feminino', 'diretoria', 'other')),
    logo_url TEXT NOT NULL,
    alt_text VARCHAR(180),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE (context, is_active)
);

CREATE TABLE forum_categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL UNIQUE,
    description TEXT,
    is_public BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE forum_topics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id UUID REFERENCES forum_categories(id) ON DELETE SET NULL,
    author_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    title VARCHAR(180) NOT NULL,
    content TEXT NOT NULL,
    visibility VARCHAR(30) NOT NULL DEFAULT 'public' CHECK (visibility IN ('public', 'members', 'admin')),
    status VARCHAR(30) NOT NULL DEFAULT 'published' CHECK (status IN ('draft', 'published', 'hidden', 'closed')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
```

```
);

CREATE TABLE forum_posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    topic_id UUID NOT NULL REFERENCES forum_topics(id) ON DELETE CASCADE,
    author_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    content TEXT NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'published' CHECK (status IN ('published', 'hidden', 'deleted')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE news_posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    author_user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    title VARCHAR(180) NOT NULL,
    summary TEXT,
    content TEXT NOT NULL,
    image_url TEXT,
    event_date TIMESTAMP,
    event_location VARCHAR(180),
    show_on_home BOOLEAN NOT NULL DEFAULT TRUE,
    show_on_forum BOOLEAN NOT NULL DEFAULT TRUE,
    is_featured BOOLEAN NOT NULL DEFAULT FALSE,
    status VARCHAR(30) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published', 'hidden', 'archived')),
    published_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_cash_movements_account_date ON cash_movements(account_id, occurred_at);
CREATE INDEX idx_cash_movements_source ON cash_movements(payment_id, order_id, monthly_fee_id);
CREATE INDEX idx_products_partner_sale_mode ON products(partner_id, sale_mode);
CREATE INDEX idx_product_batches_status_dates ON product_batches(status, sales_start_at, sales_end_at);
CREATE INDEX idx_carts_user_status ON carts(user_id, status);
CREATE INDEX idx_forum_topics_status ON forum_topics(status, created_at);
```

```
CREATE INDEX idx_news_posts_publication ON news_posts(status, show_on_home, show_on_forum, published_at);
```

10. Organização inicial do projeto

Parte	Estrutura sugerida
Frontend Next.js	src/app, src/components, src/styles, src/services, src/hooks, src/utils
Backend Node.js	src/modules/auth, athletes, teams, finance, store, tryouts, reports, users
Banco	migrations, seeds, schema, indexes
Camadas do back-end	routes/controllers, services, repositories, validators, middlewares
Segurança	middleware de autenticação, middleware de autorização por perfil, validação de entrada e logs
Novos módulos do back-end	cash, partners, cart, favorites, forum, news, settings, uploads, appointments, inventory
Arquivos/anexos	Serviço de upload para logos, comprovantes, imagens de produtos, notícias e documentos financeiros

11. Próximos passos recomendados

1. Criar protótipo visual no Figma ou diretamente no Next.js usando os wireframes deste documento.
2. Definir quais campos são realmente obrigatórios no cadastro de atleta e inscrição pública.
3. Criar repositório com front-end e back-end separados ou monorepo organizado.
4. Implementar autenticação e permissões antes do painel administrativo completo.
5. Começar pelo MVP: site público, inscrição, atletas, planos/mensalidades e produtos básicos.
6. Depois integrar pagamento PIX/Mercado Pago, notificações automáticas e relatórios avançados.
7. Definir o plano de categorias do Caixa Fênix antes de iniciar registros financeiros reais.
8. Definir política de comprovantes: tipos aceitos, tamanho máximo, quem pode visualizar e por quanto tempo manter.
9. Definir regras de parceiros: aprovação, repasse, responsabilidade por entrega/atendimento e contato oficial.
10. Prototipar a loja com carrinho, favoritos e fluxo específico para remessa de uniformes antes de implementar pagamento automático.
11. Criar regras de moderação para fórum e notícias, separando comunicado oficial de comentários comuns.